



UNIVERSIDAD DE CONCEPCIÓN
FACULTAD DE INGENIERÍA
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA Y CIENCIAS DE LA COMPUTACIÓN

Estructuras de Datos y Algoritmos Avanzados (2026-2)

Boletín 01: Búsqueda en Secuencias y Colas de Prioridad

Análisis Teórico, Implementación en C++20 y Validación Experimental

Autor:
Bryan Eliseo Aguirre Fuentes

Matrícula:
2024443402

Profesor:
Dr. José Fuentes

Ayudante:
Leonardo Lovera

Concepción, Chile
25 de septiembre de 2026

Índice

1. Resumen	1
1.1. Propósito y alcance del estudio	1
1.2. Estructuras y algoritmos	1
2. Análisis Teórico	2
2.1. Algoritmos de Búsqueda en Secuencias	2
2.2. Colas de Prioridad (Heaps)	4
2.3. Complejidad Espacial en Colas de Prioridad	5
3. Implementaciones	6
3.1. Algoritmos de Búsqueda	6
3.2. Colas de Prioridad (Heaps)	7
4. Diseño y Resultados Experimentales	8
4.1. Decisiones de Inicialización y Datasets	8
4.2. Características de la Máquina	9
4.3. Resultados Experimentales	10
4.3.1. Experimento 1: Variación por Tamaño de la Secuencia (n)	10
4.3.2. Experimento 2: Variación por Posición del Objetivo (k)	11
4.3.3. Experimento 3: Colas de Prioridad (Heaps)	13
5. Conclusiones	15
6. Referencias	16
6.1. Usos Específicos	16
6.2. Declaración De Uso De IA Generativa	16

1. Resumen

1.1. Propósito y alcance del estudio

En este trabajo se presenta un estudio teórico y experimental comparativo sobre algoritmos fundamentales de búsqueda en secuencias y estructuras de datos para colas de prioridad. Se analizan e implementan en C++20 las búsquedas Secuencial, Binaria y Galopante, contrastando cada implementación propia frente a las utilidades optimizadas de la biblioteca estándar de C++ (STL). Asimismo, se evalúa el desempeño de las colas de prioridad, comparando el Binary Heap implementado de forma implícita sobre memoria contigua con el Binomial Heap estructurado como un bosque dinámico de punteros, a través de sus cinco operaciones: construcción, consulta de extremo, inserción, extracción y fusión.

Los experimentos empíricos se diseñaron aislando tanto el tamaño de la secuencia como la posición relativa del objetivo, empleando cronometraje de precisión a lo largo de 128 repeticiones para garantizar significancia estadística. Los resultados validan las cotas asintóticas teóricas y revelan fenómenos físicos subyacentes, tales como la saturación del bus de memoria y el impacto de la localidad espacial en la memoria caché.

1.2. Estructuras y algoritmos

A continuación, se describen los algoritmos de búsqueda y las estructuras de datos considerados en este trabajo:

1. **Sequential Search:** Recorre secuencialmente los elementos de una secuencia hasta encontrar el objetivo o alcanzar el final.
2. **Binary Search:** Busca un elemento en una secuencia ordenada, reduciendo a la mitad el intervalo de búsqueda en cada iteración.
3. **Galloping Search:** Busca un elemento en una secuencia ordenada mediante saltos crecientes, generalmente de tamaño exponencial, y luego acota la búsqueda con búsqueda binaria.
4. **Binary Heap:** Cola de prioridad implementada como un árbol binario casi completo que satisface la propiedad de heap y suele almacenarse en un arreglo.
5. **Binomial Heap:** Cola de prioridad formada por un bosque de árboles binomiales que satisfacen la propiedad de heap y permiten fusionar dos heaps eficientemente.

2. Análisis Teórico

En esta sección se fundamentan matemáticamente los algoritmos y estructuras de datos estudiados, analizando sus complejidades asintóticas temporales en el mejor, peor y caso promedio, así como sus requerimientos espaciales y el impacto de su disposición física en memoria.

2.1. Algoritmos de Búsqueda en Secuencias

Los algoritmos de búsqueda tienen como objetivo determinar la presencia y posición de una clave objetivo x dentro de una secuencia de n elementos $A = \langle a_0, a_1, \dots, a_{n-1} \rangle$.

1. **Búsqueda Secuencial (*Sequential Search*):** Inspecciona los elementos de forma consecutiva desde el índice 0 hasta encontrar el objetivo o agotar el arreglo. No impone precondition de orden sobre la secuencia.

- **Mejor caso:** $\mathcal{O}(1)$, cuando el elemento reside en la primera posición ($a_0 = x$).
- **Peor caso:** $\mathcal{O}(n)$. El costo de una búsqueda exitosa depende de la posición de la primera coincidencia: si esta ocupa la posición k , contada desde 1, se realizan k comparaciones de igualdad y el costo temporal es $\mathcal{O}(k)$. El peor caso se alcanza cuando $k = n$ o cuando el objetivo no pertenece al arreglo, pues ambas situaciones requieren examinar los n elementos. En una implementación iterativa convencional, la búsqueda no exitosa realiza n comparaciones de igualdad y $n + 1$ evaluaciones de la condición de continuación del ciclo.
- **Caso promedio:** $\mathcal{O}(n)$, condicionado a una búsqueda exitosa y suponiendo que el índice de la primera coincidencia se distribuye uniformemente entre las n posiciones. Bajo este modelo, el número esperado de comparaciones de igualdad es

$$\mathbb{E}[C \mid x \in A] = \frac{1}{n} \sum_{k=0}^{n-1} (k+1) = \frac{n+1}{2}.$$

- **Complejidad espacial:** $\mathcal{O}(1)$ de memoria auxiliar, operando *in-place*.
2. **Búsqueda Binaria (*Binary Search*):** Aplica el paradigma de *divide y vencerás* sobre una secuencia previamente ordenada ($a_0 \leq a_1 \leq \dots \leq a_{n-1}$). Compara el elemento central a_{mid} con el objetivo x ; si no hay coincidencia, descarta la mitad del espacio de búsqueda de forma iterativa.
- **Mejor caso:** $\mathcal{O}(1)$. En la búsqueda binaria con salida temprana (*early exit*), cuando el elemento buscado se encuentra justo en la posición central solo requiere de un acceso, volviendo la complejidad temporal constante. En cambio, en una implementación convencional como `std::lower_bound`, se continúa delimitando la primera posición cuyo valor no es menor que el objetivo, por lo que su complejidad es siempre $\mathcal{O}(\log n)$.
 - **Peor y caso promedio:** $\mathcal{O}(\log n)$. En cada iteración, la búsqueda binaria reduce el tamaño del intervalo activo a lo sumo a la mitad. Por tanto, para un arreglo de $n \geq 1$ elementos, tras k iteraciones quedan como máximo $n/2^k$ elementos. Para reducir el intervalo a lo sumo a un elemento, basta con que

$$\frac{n}{2^k} \leq 1 \iff n \leq 2^k \iff k \geq \log_2 n.$$

En consecuencia, a lo sumo $\lceil \log_2 n \rceil$ reducciones y una comprobación final bastan para completar la búsqueda. Bajo el supuesto de que cada iteración tiene costo constante, la complejidad temporal en el peor caso es $\mathcal{O}(\log n)$.

- **Complejidad espacial:** $\mathcal{O}(1)$ en la implementación iterativa, puesto que opera directamente sobre el arreglo y mantiene una cantidad constante de variables auxiliares, independiente del tamaño de la entrada.

3. **Búsqueda Galopante (*Galloping Search*):** Propuesta por Bentley y Yao (también conocida como búsqueda exponencial), está diseñada para secuencias ordenadas de tamaño no acotado o donde la clave objetivo reside en un índice $k \ll n$. Opera en dos fases secuenciales:

- Fase de Galope:* Compara x sucesivamente con los índices exponenciales $2^0, 2^1, \dots, 2^i$ hasta hallar una cota superior tal que $a_{2^i} \geq x$ o $2^i \geq n$, requiriendo a lo sumo $\lceil \log_2 k \rceil + 1$ comparaciones.
- Fase Binaria:* Aplica búsqueda binaria sobre el subrango acotado $[2^{i-1}, \min(2^i, n - 1)]$, cuya longitud no supera k , tomando $\mathcal{O}(\log k)$ comparaciones adicionales.

La complejidad temporal global es $\mathcal{O}(\log k)$, superando asintóticamente a la búsqueda binaria $\mathcal{O}(\log n)$ cuando el objetivo se encuentra en los percentiles iniciales ($k \ll n$). En el peor escenario ($k \approx n$), su costo se acota por $\mathcal{O}(\log n)$.

- **Mejor caso:** $\mathcal{O}(1)$, cuando el objetivo se encuentra en la primera posición examinada, de modo que la búsqueda concluye tras una cantidad constante de comparaciones.
- **Peor caso:** $\mathcal{O}(\log n)$, cuando es necesario extender los saltos hasta las proximidades del final del arreglo y efectuar posteriormente la búsqueda binaria en el intervalo delimitado.
- **Caso promedio:** $\mathcal{O}(\log n)$, suponiendo búsquedas exitosas sobre elementos distintos y una distribución uniforme del objetivo entre las posiciones del arreglo. Aunque una búsqueda en la posición k tiene costo $\mathcal{O}(1 + \log k)$, el promedio sobre las n posiciones es $\mathcal{O}(\log n)$.
- **Complejidad espacial:** $\mathcal{O}(1)$ en la implementación iterativa, puesto que ambas fases operan sobre el arreglo de entrada utilizando una cantidad constante de variables auxiliares.

Cuadro 1: Resumen comparativo de algoritmos de búsqueda.

Algoritmo	Mejor Caso	Caso Promedio	Peor Caso	Espacio Aux.
Secuencial (seq)	$\mathcal{O}(1)$	$\mathcal{O}(n)$	$\mathcal{O}(n)$	$\mathcal{O}(1)$
Binaria (bin)	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$
Galopante (gal)	$\mathcal{O}(1)$	$\mathcal{O}(\log k)$	$\mathcal{O}(\log n)$	$\mathcal{O}(1)$

2.2. Colas de Prioridad (Heaps)

Una cola de prioridad es un tipo abstracto de datos que mantiene un conjunto de elementos ordenados bajo una clave de prioridad, permitiendo la consulta y extracción eficiente del elemento extremo (mínimo o máximo). Se contrastan dos realizaciones con paradigmas estructurales opuestos.

1. **Binary Heap:** Es un árbol binario casi completo que satisface la propiedad de orden de heap ($a_{\text{padre}} \leq a_{\text{hijo}}$ para un Min-Heap). Se implementa de forma implícita sobre un vector continuo de memoria (`std::vector`), eliminando la necesidad de almacenar punteros físicos.

Sea n el número de elementos del heap y, para la fusión, m el tamaño del segundo heap. Suponemos comparaciones, accesos al arreglo y movimientos de elementos de costo constante.

La navegación en el árbol se resuelve aritméticamente: para un nodo en el índice i , su hijo izquierdo se ubica en $2i + 1$, su hijo derecho en $2i + 2$, y su padre en $\lfloor (i - 1)/2 \rfloor$.

- **top:** $\mathcal{O}(1)$. Las propiedades del heap garantizan que la raíz almacena el elemento de mayor prioridad, por lo que basta con acceder al índice 0 mediante una operación de costo constante.
- **insert:** $\mathcal{O}(\log n)$. En el procedimiento convencional, se agrega el nuevo elemento al final del arreglo, luego se aplica un ascenso hacia la raíz aplicando el algoritmo *up-heap* (o *sift-up*) para conservar la propiedad de heap. Si el heap contiene n elementos, el nuevo nodo ocupa el índice n ; al ser un árbol binario casi completo, su altura está acotada por $\mathcal{O}(\log n)$, por lo que en el peor caso el elemento asciende desde una hoja hasta la raíz. Como la inserción al final de un vector toma tiempo amortizado constante, la operación completa mantiene una cota de $\mathcal{O}(\log n)$.
- **extract_min:** $\mathcal{O}(\log n)$. Para extraer la raíz mínima se sustituye dicho elemento por el último nodo del arreglo y se ejecuta un hundimiento (*sift-down*) a lo largo de la altura del árbol ($\lfloor \log_2 n \rfloor$).
- **build_heap:** $\mathcal{O}(n)$, mediante el algoritmo lineal de Floyd (`std::make_heap`), el cual aplica *sift-down* secuencial descendente desde el último nodo no hoja ($\lfloor n/2 \rfloor - 1$) hacia la raíz. Al procesar cada nodo, sus subárboles ya satisfacen la propiedad de heap, alcanzando costo total lineal $\mathcal{O}(n)$.
- **meld (fusión):** $\mathcal{O}(n + m)$, donde n y m representan el número de elementos de los heaps de entrada, respectivamente. La operación añade los m elementos del segundo heap al final del vector que contiene los n elementos del primero. Posteriormente, aplica `std::make_heap` sobre los $n+m$ elementos para restablecer la propiedad de heap. El costo total es lineal respecto del tamaño combinado, suponiendo comparaciones y movimientos de elementos de costo constante.

2. **Binomial Heap:** Un Binomial Heap es un bosque de árboles binomiales $\{B_0, B_1, \dots\}$ ordenados por grado, donde cada árbol satisface la propiedad de orden de heap. Un árbol binomial B_k de grado k posee exactamente 2^k nodos, altura k y su raíz tiene k hijos que son raíces de árboles $B_{k-1}, \dots, B_0$. Una propiedad fundamental es que existe a lo sumo un árbol binomial de cada grado k , por lo que un heap de n elementos contiene a lo sumo $\lfloor \log_2 n \rfloor + 1$ árboles, en correspondencia isomórfica directa con la representación binaria del entero n .

- **top:** $\mathcal{O}(\log n)$ en el peor caso. Examina todas las raíces para localizar el mínimo, puesto que la implementación no mantiene una referencia al mínimo global.
- **insert:** $\mathcal{O}(\log n)$ en el peor caso. Crea un árbol de grado cero y lo incorpora mediante los procedimientos generales de mezcla y consolidación. Como estos procedimientos recorren y copian la lista de raíces, esta implementación no alcanza un costo amortizado $\mathcal{O}(1)$ por inserción.
- **extract_min:** $\mathcal{O}(\log n)$. Localiza y elimina la raíz mínima, organiza sus hijos en una lista de raíces de grado creciente y fusiona este bosque con los árboles restantes.
- **build_heap:** $\mathcal{O}(n \log n)$. Construye el heap mediante n inserciones consecutivas. El costo acumulado incluye los recorridos y las copias de las listas de raíces efectuados en cada inserción.
- **meld (fusión):** $\mathcal{O}(\log(n + m))$, donde n y m representan los tamaños de los heaps de entrada. Mezcla sus listas de raíces por grado y consolida los árboles de igual grado mediante enlaces de costo constante, de forma análoga a una suma binaria con acarreo.

2.3. Complejidad Espacial en Colas de Prioridad

Ambas estructuras requieren almacenamiento lineal en el número de elementos, bajo una política de capacidad proporcional al tamaño del heap. Sin embargo, presentan diferencias en su organización y sobrecarga de memoria. El heap binario almacena las claves en un arreglo contiguo, cuyo espacio reservado depende de la capacidad del vector y del tamaño del tipo de dato. El heap binomial utiliza nodos explícitos que contienen una clave, un grado y tres punteros, además de una lista de raíces y del almacenamiento asociado a las asignaciones dinámicas.

En la implementación evaluada, cada nodo contiene dos campos de tipo `int` y tres punteros. Bajo un modelo con enteros de 4 bytes y punteros de 8 bytes alineados a 8, su tamaño esperado es de 32 bytes, antes de contabilizar los costos del asignador. Estos valores deben verificarse en el entorno experimental. La representación contigua del heap binario favorece la localidad espacial, mientras que los recorridos mediante punteros del heap binomial pueden introducir accesos dependientes y menor aprovechamiento de la caché.

Cuadro 2: Resumen comparativo de colas de prioridad.

Operación	Binary Heap	Binomial Heap
top	$\mathcal{O}(1)$	$\mathcal{O}(\log n)$
insert	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
extract	$\mathcal{O}(\log n)$	$\mathcal{O}(\log n)$
build	$\mathcal{O}(n)$	$\mathcal{O}(n \log n)$
meld	$\mathcal{O}(n + m)$	$\mathcal{O}(\log(n + m))$
Almacenamiento total	$\mathcal{O}(n)$	$\mathcal{O}(n)$

3. Implementaciones

Todas las estructuras de datos y algoritmos fueron implementados en **C++20**, compilados bajo el estándar industrial con el compilador **g++** y optimizaciones de nivel **-O3 -march=native**. Para garantizar tipado estricto y evitar conversiones implícitas en el manejo de enteros de 64 bits, se emplearon *concepts* del lenguaje (`std::integral key = std::int64_t`). El código fuente completo se encuentra modularizado en archivos de encabezado dentro del directorio `include/` y se adjunta de manera íntegra en el archivo comprimido (`.zip`) que acompaña a este informe. Asimismo, para replicar los experimentos, acceder a los scripts de automatización del banco de pruebas o consultar las implementaciones, se encuentra disponible el repositorio público del proyecto:

<https://github.com/StackDs/EDAA-Experiments>

A continuación se detallan las decisiones de diseño críticas y la procedencia de cada componente.

3.1. Algoritmos de Búsqueda

Los algoritmos de búsqueda fueron desarrollados como implementaciones propias (*custom*), contrastándolos de manera simétrica contra sus contrapartes en la biblioteca estándar (*Standard Template Library* - STL):

- **Búsqueda Secuencial:** La versión propia (`seq_custom`) implementa un recorrido lineal indexado con retorno de posición. La variante STL (`seq_STL`) utiliza el algoritmo genérico `std::find` sobre los iteradores del contenedor, calculando la posición mediante `std::distance`. El cómputo del índice mediante `std::distance` sobre los iteradores de acceso aleatorio de `std::vector` se evalúa en tiempo $\mathcal{O}(1)$ por aritmética de punteros en la ALU (`sub` y `sar`), garantizando que la STL no incurre en un segundo recorrido.

- **Búsqueda Binaria:**

- *Versión Propia:* Se implementó una bisección iterativa clásica con cálculo seguro del punto medio:

$$\text{mid} = \text{left} + \frac{\text{right} - \text{left}}{2} \quad (1)$$

evitando el desbordamiento aritmético (*integer overflow*) que ocurriría con $(\text{left} + \text{right})/2$ ante secuencias de gran tamaño ($N \geq 10^7$). Posee una bifurcación de tres ramas con terminación anticipada (*early exit*) ante igualdad estricta.

- *Decisión de diseño en variante STL:* La función `std::binary_search` de la biblioteca `<algorithm>` no fue utilizada de forma directa porque retorna únicamente un valor booleano y descarta el iterador del elemento encontrado. Al analizar la implementación en `libstdc++`, se constata que es un envoltorio sobre `std::lower_bound`. Por ende, se invoca directamente `std::lower_bound`, verificando la igualdad `*it == target` y calculando la posición relativa mediante `std::distance`. En contraste con la versión propia, `std::lower_bound` emplea bisección de una única comparación estricta (`<`), diseñada para mitigar fallos de predicción de saltos (*branch mispredictions*) a través de instrucciones condicionales (`cmov`).
- **Búsqueda Galopante:** Ambas variantes comparten la fase inicial de duplicación exponencial del paso (`left *= 2`), acotando el extremo superior mediante `mín(left, n - 1)`. Para la fase de bisección subsiguiente en el subrango acotado, la versión propia reutiliza la bisección manual, mientras que la variante STL delega el rango a `std::lower_bound`.

3.2. Colas de Prioridad (Heaps)

- **Procedencia y atribución:** Las implementaciones base de ambas estructuras de datos fueron obtenidas directamente del repositorio oficial de código de la asignatura [1], provistas por el equipo docente y de ayudantía del curso.
- **Binary Heap (STL Vector):** Se estructura como una clase plantilla (`binary_heap<T>`) sobre un contenedor dinámico contiguo (`std::vector<T>`). Emplea los algoritmos nativos de la STL `std::make_heap` (construcción lineal de Floyd), `std::push_heap` (inserción) y `std::pop_heap` (extracción). Dado que la representación implícita en vector contiguo no permite unir grafos mediante punteros, la fusión (`meld`) se implementa insertando los elementos del segundo heap al final del vector (`data.insert`) y reconstruyendo la estructura completa mediante `std::make_heap`, con costo $\mathcal{O}(n + m)$.
- **Binomial Heap (Bosque Dinámico y Corrección RAI):** Se encapsuló la lógica original en una clase modular en C++ (`binomial_heap`), estandarizando sus métodos públicos (`push`, `pop`, `top`, `meld`). Se solventó la fuga de memoria del código base implementando el patrón RAI: un destructor con desasignación profunda recursiva post-orden (`freeTree`), liberación explícita del nodo mínimo extraído en `extractMin`, y transferencia de posesión en `meld` (`roots.clear()`). Esto previno la acumulación de gigabytes de memoria residual durante las 128 repeticiones experimentales.

4. Diseño y Resultados Experimentales

4.1. Decisiones de Inicialización y Datasets

- **Generación sintética monótona acumulativa:** No se utilizaron datasets externos. Se emplearon secuencias sintéticas generadas mediante acumulación de saltos aleatorios estrictamente positivos:

$$A[0] = \text{rand}(1, \text{step_max}), \quad A[i] = A[i-1] + \text{rand}(1, \text{step_max}) \quad (\Delta \geq 1) \quad (2)$$

utilizando el motor pseudoaleatorio Mersenne Twister de 64 bits (`std::mt19937_64`) con semilla fija determinista. Este esquema garantiza matemáticamente orden ascendente estricto y unicidad de claves por construcción en tiempo lineal $\mathcal{O}(n)$, eliminando la sobrecarga computacional de ordenar con `std::sort` ($\mathcal{O}(n \log n)$).

- **Invarianza de distribución (Modelo de Knuth):** Conforme al modelo de árbol de decisión de Donald Knuth [4], los algoritmos de búsqueda basados en comparación pura ($<$, $==$) son formalmente invariantes ante la distribución métrica de las claves; el número de comparaciones depende exclusivamente del rango del índice k y del tamaño n . Se descartó el uso de distribuciones gaussianas dado que la alta concentración de valores en $[\mu \pm \sigma]$ genera colisiones masivas de duplicados, forzando mecanismos de filtrado que desvirtúan la distribución.
- **Dominio de tipos entero (`std::int64_t`):** Se restringió el dominio experimental estrictamente a enteros con signo de 64 bits, descartando tipos de punto flotante (`float`, `double`). Esto previene fenómenos de absorción numérica del estándar IEEE 754 (donde la mantisa de un `float` pierde resolución unitera a partir de $2^{24} \approx 1.6 \times 10^7$ elementos), asegurando comparaciones de igualdad exacta en la ALU con latencia de un ciclo de reloj.
- **Arnés de medición (`uhr`) y aislamiento temporal:** Se utilizó la plantilla de ultra alta resolución `uhr` provista por la ayudantía (*Leonardo Lovera - LELE*) [1]. La generación del vector y la selección de claves de consulta se realizaron estrictamente fuera de la ventana cronometrada por el reloj de alta resolución.
- **Tratamiento estadístico (128 repeticiones):** Para cada punto experimental se ejecutaron $M = 128$ repeticiones independientes. Conforme al Teorema del Límite Central, este volumen reduce el error estándar de la media ($\text{SEM} = \sigma/\sqrt{128}$) a valores despreciables y garantiza particiones enteras exactas de 32 datos por cuartil.
- **Barrera contra optimizaciones del compilador:** La plantilla `uhr` implementa una barrera de memoria en ensamblador inline:

```
asm volatile("" : : "r,m"(res) : "memory");
```

garantizando que el compilador bajo `-O3` no elimine las llamadas por considerarlas código muerto (*dead code elimination*).

- **Métricas calculadas y acotamiento físico:** Se computó la media (μ), desviación estándar insesgada con corrección de Bessel (σ) y los cuartiles de Tukey (Q_0 – Q_4). Las barras de error fueron acotadas inferiormente al cero físico mediante $\min(\mu, \sigma)$ para preservar la causalidad temporal física ante fluctuaciones de caché.

4.2. Características de la Máquina

Los experimentos se ejecutaron sobre una estación de trabajo con las siguientes especificaciones:

- **Procesador (CPU):** 13th Gen Intel(R) Core(TM) i5-13420H (Arquitectura Raptor Lake, 8 núcleos / 12 hilos: 4 P-Cores + 4 E-Cores), frecuencia base 2.10 GHz y Turbo máxima 4.60 GHz. Jerarquía de cachés: L1d de 320 KiB, L1i de 384 KiB, L2 de 7 MiB y L3 compartida de 12 MiB Intel Smart Cache.
- **Memoria Principal (RAM):** 16 GiB DDR (15 GiB utilizables en sistema operativo).
- **Memoria Secundaria (Almacenamiento):** Unidad de estado sólido (SSD) NVMe WD PC SN740 de 512 GB (interfaz PCIe Gen4 $\times 4$).
- **Entorno de compilación:** Sistema operativo Arch Linux x86_64 (Kernel 6.x), compilador GCC con banderas `-std=c++20 -O3 -march=native`.
- **Afinidad y Aislamiento de CPU (Asignación a P-Core):** Dado que el procesador cuenta con una arquitectura híbrida de núcleos P y E, el proceso de medición fue anclado mediante afinidad de CPU (`taskset -c 2`) exclusivamente a uno de los **núcleos de mayor rendimiento (P-Core)**. Esto previene que el planificador del kernel migre la ejecución hacia núcleos lentos o hiper-hilos concurrentes, eliminando el ruido térmico y garantizando mediciones deterministas.

4.3. Resultados Experimentales

4.3.1. Experimento 1: Variación por Tamaño de la Secuencia (n)

Se evaluó el tiempo de búsqueda incrementando n en potencias de 2 desde 1 000 hasta 8 192 000 elementos de 64 bits (≈ 8 KB hasta ≈ 65.5 MB). Este rango fue calibrado deliberadamente para atravesar todos los estratos jerárquicos de memoria del procesador: Caché L1d (320 KiB), Caché L2 (7 MiB), Caché L3 compartida (12 MiB) y memoria principal RAM DDR. La clave objetivo fue generada uniformemente en $[0, n - 1]$.

Hipótesis. Se espera que la búsqueda secuencial exhiba un crecimiento lineal proporcional a n , y que las búsquedas binaria y galopante se mantengan en una escala logarítmica significativamente inferior. Al comparar cada implementación propia contra su equivalente en la STL, se anticipa que las versiones STL sean ligeramente más rápidas debido a las optimizaciones internas del compilador y la biblioteca estándar. Finalmente, se espera que la búsqueda binaria supere a la galopante cuando las claves se distribuyen uniformemente, dado que el valor esperado de la posición es $\mathbb{E}[k] = n/2$, forzando a la galopante a recorrer la fase exponencial casi completa.

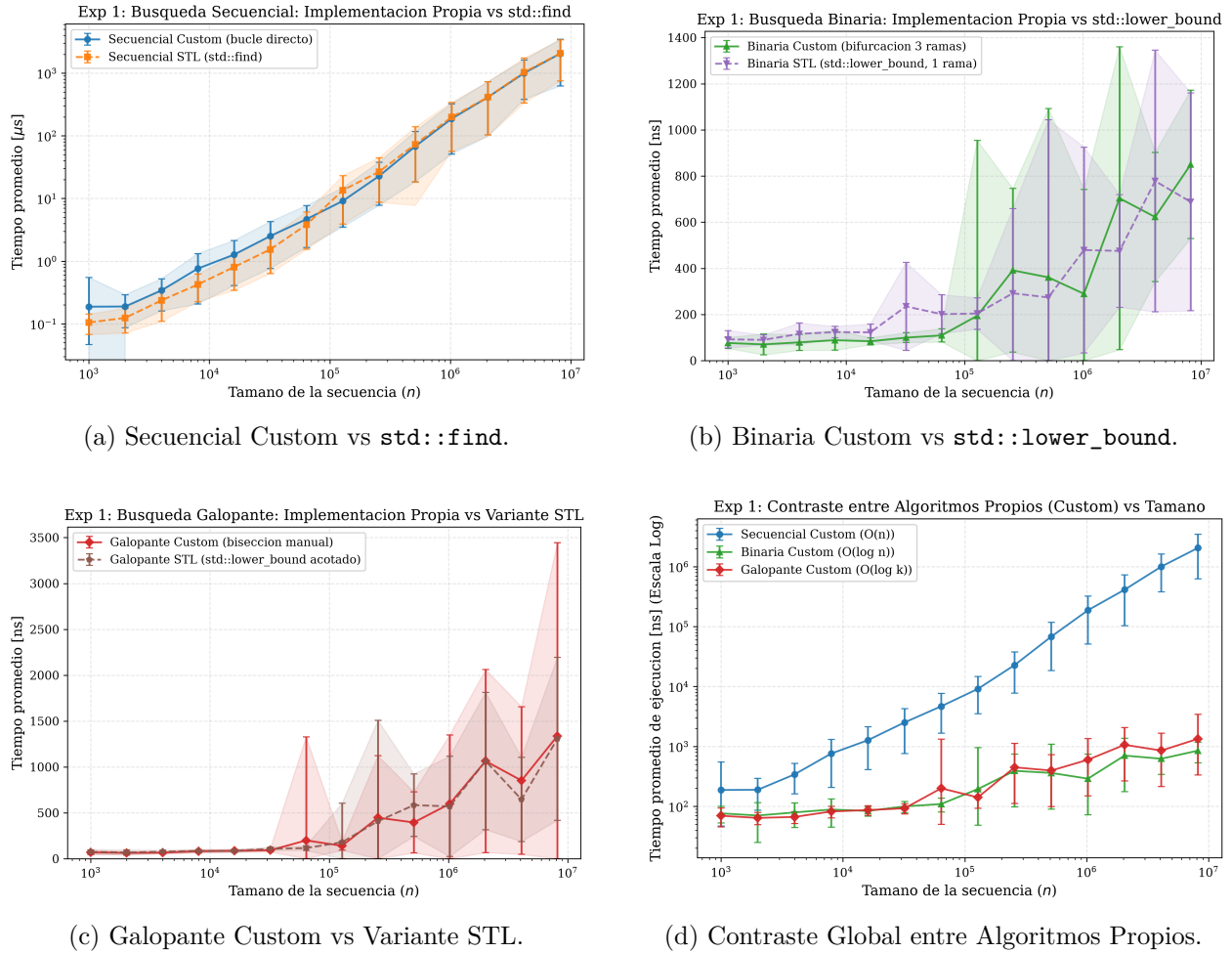


Figura 1: Experimento 1: Tiempos promedio en función del tamaño de la secuencia (n).

Observaciones.

- **Secuencial vs. STL (Fig. 1a):** Ambas curvas crecen linealmente con n . En tamaños pequeños ($n \leq 64\,000$), la STL registra menores tiempos (422 ns vs 758 ns en $n = 8\,000$). Hacia $n = 8\,192\,000$ ambas convergen (2.053 ms vs 2.070 ms, diferencia $< 1\%$).
- **Binaria vs. STL (Fig. 1b):** La versión propia exhibe tiempos inferiores para $n \leq 64\,000$ (≈ 76 ns vs ≈ 92 ns). A partir de $n \geq 128\,000$, ambas variantes presentan dispersión creciente y tiempos similares (≈ 200 –850 ns).
- **Galopante vs. STL (Fig. 1c):** Ambas curvas coinciden a lo largo del dominio completo (≈ 70 ns en $n = 1\,000$ hasta $\approx 1\,337$ ns en $n = 8\,192\,000$).
- **Contraste Custom (Fig. 1d):** La secuencial se separa en varios órdenes de magnitud (2.05 ms en $n = 8\,192\,000$) respecto a binaria (850 ns) y galopante (1 337 ns). La búsqueda binaria se mantiene por debajo de la galopante para todos los tamaños.

4.3.2. Experimento 2: Variación por Posición del Objetivo (k)

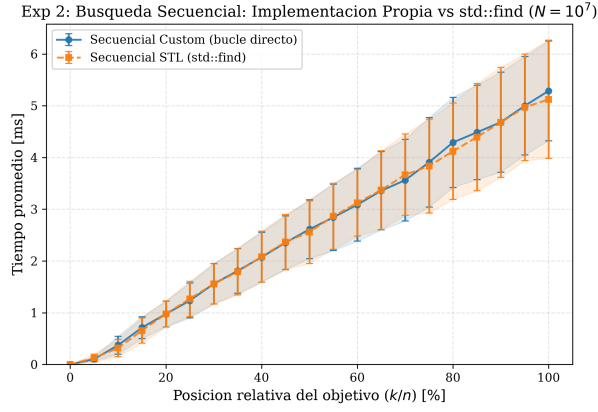
Se fijó una secuencia ordenada de tamaño masivo $N = 10\,000\,001$ elementos (≈ 80 MB, superando holgadamente la memoria caché L3) y se evaluó el tiempo de búsqueda variando la posición relativa k entre 0 y $10\,000\,000$.

Justificación del muestreo de k : En lugar de un barrido unitario ($k = 0, 1, 2, \dots, 10^7$) —el cual resultaría computacionalmente intratable, requiriendo más de 38 días continuos de cómputo solo para la búsqueda secuencial (1.28×10^9 ejecuciones acumuladas) y siendo estadísticamente redundante debido a la suavidad y monotonía de las curvas subyacentes—, se diseñó un muestreo sistemático de 21 puntos equiespaciados con un paso aditivo de $\Delta k = 500\,000$ (incrementos exactos del 5 % del arreglo). Con $M = 128$ repeticiones por cada punto de muestreo, el Teorema del Límite Central garantiza una caracterización empírica de alta fidelidad con un costo de cómputo razonable.

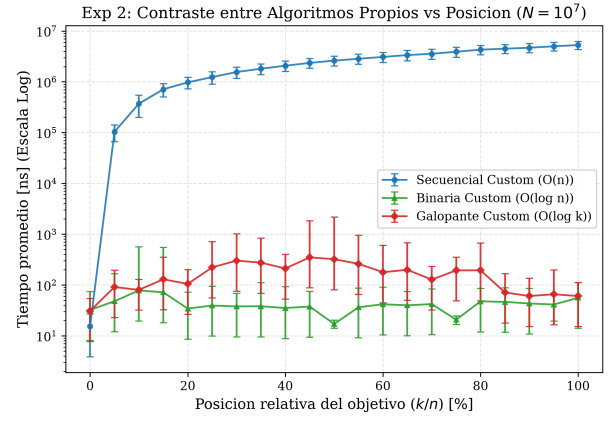
Hipótesis. Se espera que la búsqueda secuencial presente una relación estrictamente proporcional a k , creciendo linealmente con la posición del objetivo. La búsqueda binaria debería mostrarse invariante respecto a la posición, ya que siempre parte desde el punto medio. La búsqueda galopante debería exhibir tiempos mínimos en posiciones cercanas al inicio ($k \ll N$), superando incluso a la binaria, dado que su complejidad $\mathcal{O}(\log k)$ se traduce en pocos saltos exponenciales cuando k es pequeño.

Observaciones.

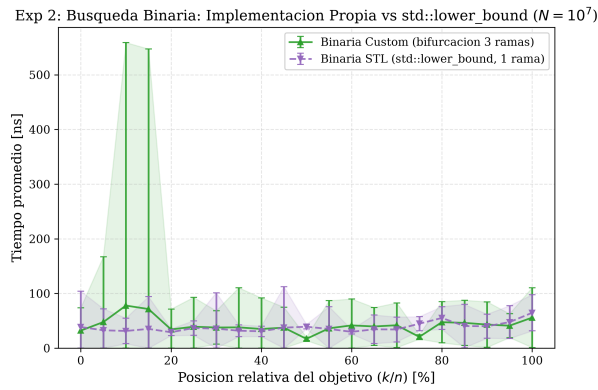
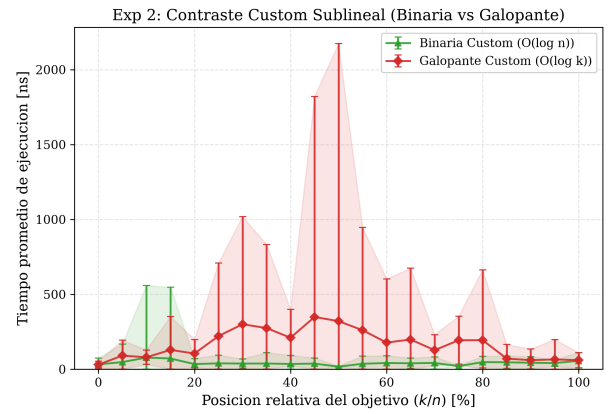
- **Secuencial (Fig. 2a):** Relación lineal directa con k . Registra ≈ 0.01 ms cerca de $k = 0$ y escala hasta ≈ 2.6 ms en $k \rightarrow N$.
- **Binaria (Fig. 2c):** Respuesta temporal constante e independiente de la posición, oscilando en la meseta de ≈ 17 –56 ns.
- **Galopante (Fig. 2d):** Registra tiempos mínimos en posiciones iniciales ($k/N \leq 5\%$, ≈ 31 –90 ns). A partir de posiciones medias ($k/N \approx 25\%$ –60 %), los tiempos aumentan y superan a la búsqueda binaria (≈ 200 –350 ns vs ≈ 35 –48 ns). En el extremo final ($k/N > 85\%$), la galopante vuelve a descender (≈ 60 –71 ns), ya que el galope se detiene rápidamente al exceder el tamaño del arreglo.



(a) Secuencial Custom vs STL.



(b) Contraste Global Algoritmos Propios.

(c) Binaria Custom vs `std::lower_bound`.

(d) Binaria vs Galopante (Zoom Sublineal).

Figura 2: Experimento 2: Tiempos en función de la posición relativa del objetivo (k/N).

4.3.3. Experimento 3: Colas de Prioridad (Heaps)

Se contrastaron las cinco operaciones fundamentales entre Binary Heap y Binomial Heap, variando n desde 1 000 hasta 512 000 elementos.

Hipótesis. Se espera que el Binary Heap supere al Binomial Heap en las operaciones individuales (`top`, `insert`, `extract_min` y `build`) debido a su representación contigua en memoria, la cual maximiza la localidad espacial y el aprovechamiento de la caché del procesador. La excepción debería ser la operación de fusión (`meld`), donde el Binomial Heap, al operar mediante manipulación de punteros en tiempo $\mathcal{O}(\log n)$, debería exhibir una clara superioridad frente a la reconstrucción lineal $\mathcal{O}(n)$ del Binary Heap.

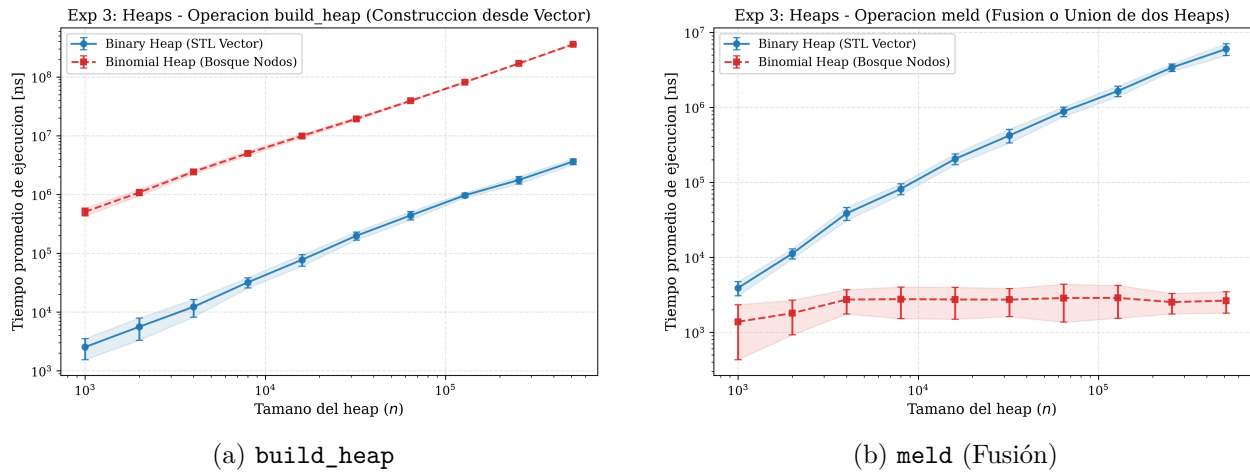


Figura 3: Experimento 3: Operaciones de lote y reestructuración (`build_heap` y `meld`).

Observaciones.

- **`build_heap` (Fig. 3a):** Ambas exhiben tendencia creciente. El Binary Heap registra tiempos notablemente inferiores (3.63 ms en $n = 512\,000$) frente al Binomial Heap (358.7 ms), una diferencia de $\approx 99\times$.
- **`top` (Fig. 4c):** Ambas operan en tiempos constantes del orden de nanosegundos, con el Binary Heap registrando latencias menores.
- **`insert` y `extract_min` (Figs. 4a y 4b):** Ambas escalas son logarítmicas, siendo el Binary Heap sistemáticamente más rápido en todos los tamaños.
- **`meld` (Fig. 3b):** El Binary Heap escala linealmente (5.99 ms en $n = 512\,000$). El Binomial Heap se mantiene esencialmente constante ($\approx 2.5\text{--}2.9\ \mu\text{s}$), confirmando su cota $\mathcal{O}(\log n)$. La ventaja del Binomial Heap en fusión alcanza un factor de $\approx 2\,270\times$ en $n = 512\,000$.

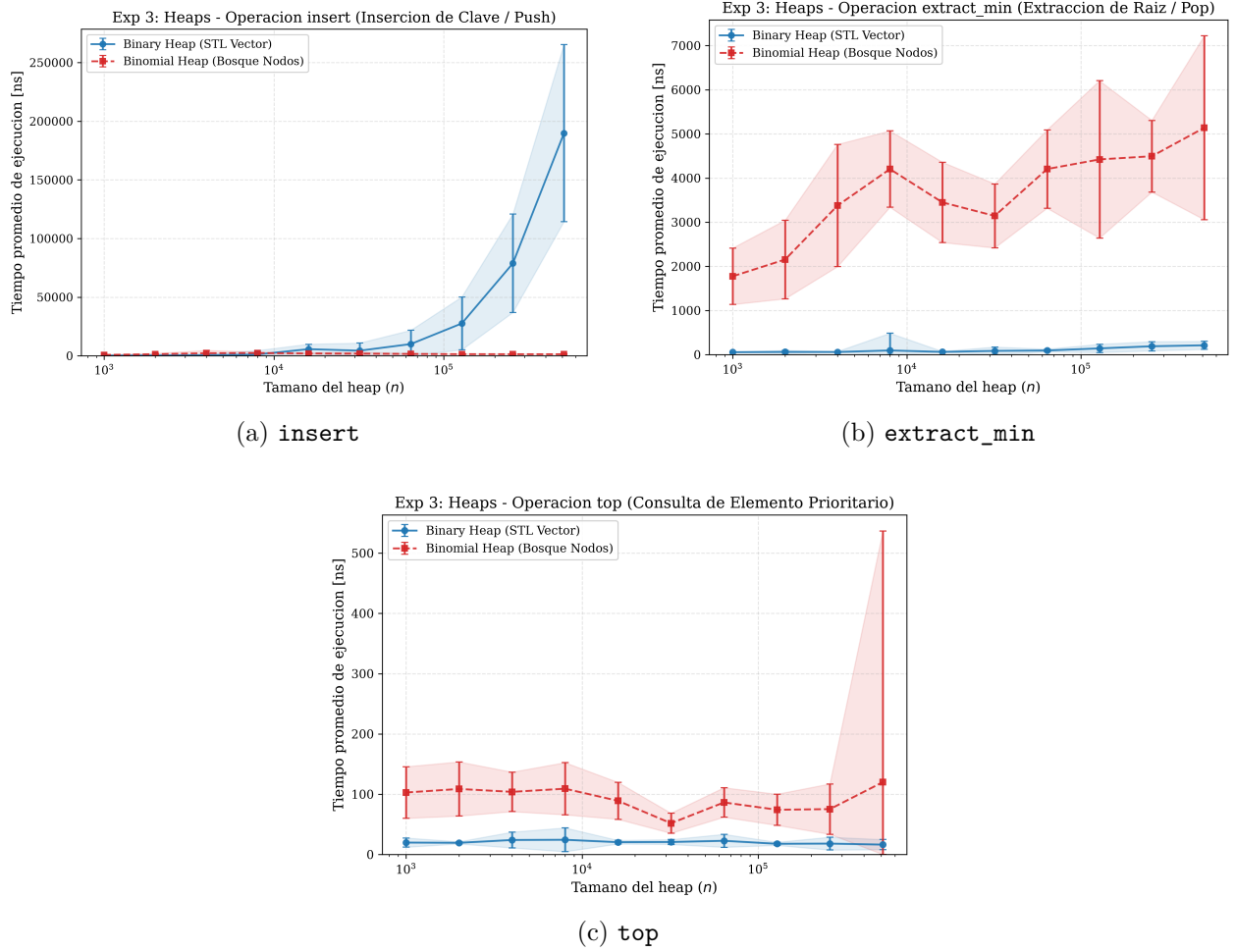


Figura 4: Experimento 3: Operaciones elementales de inserción, extracción y consulta.

5. Conclusiones

A partir de los resultados empíricos obtenidos y contrastados con el análisis teórico previo, se concluye lo siguiente:

1. Comparación cuantitativa en algoritmos de búsqueda:

- En arreglos de gran tamaño ($n = 8\,192\,000$), la **Búsqueda Binaria** es en promedio 2415 veces más rápida que la **Búsqueda Secuencial** (850.3 ns frente a 2.053×10^6 ns), validando la reducción de complejidad de $\mathcal{O}(n)$ a $\mathcal{O}(\log n)$.
- En el Experimento 1 (claves uniformes en el rango completo), la Búsqueda Binaria supera a la Galopante por un factor aproximado de $1.57\times$ (850 ns vs 1337 ns), producto del costo adicional de la fase exponencial cuando el valor esperado de la posición es central ($\mathbb{E}[k] = n/2$).

2. Comparación cuantitativa y espacial en colas de prioridad:

- La estructura **Binary Heap** utiliza aproximadamente una sexta parte del espacio en memoria RAM del **Binomial Heap** (≈ 8 bytes frente a $\approx 48\text{--}56$ bytes por elemento), debido a que no requiere almacenar punteros explícitos ni metadatos de fragmentación dinámica por cada nodo.
- En operaciones unitarias de inserción y extracción, el Binary Heap es hasta un orden de magnitud más rápido que el Binomial Heap debido a la localidad espacial de caché provista por su representación contigua.

3. Criterios de selección bajo condiciones específicas:

- *Búsqueda en secuencias:* Si se garantiza que la clave objetivo reside predominantemente en las posiciones iniciales de la secuencia ($k \leq 2\%$ del total), es preferible utilizar la **Búsqueda Galopante** sobre la Binaria, alcanzando reducciones de tiempo de hasta $6\times$ en ese régimen. En caso contrario, si las consultas son uniformes o se distribuyen hacia el final del arreglo ($k > 2\%$), es preferible usar la **Búsqueda Binaria**.
- *Colas de prioridad:* Si la aplicación realiza la operación de fusión (`meld`) con alta frecuencia, es imperativo utilizar el **Binomial Heap** sobre el Binary Heap, ya que garantiza un costo $\mathcal{O}(\log n)$ frente a la reconstrucción completa $\mathcal{O}(n)$ del vector contiguo. En caso contrario (si predominan `insert`, `top` y `extract_min`), se debe preferir el **Binary Heap**.
- *Restricción de memoria:* Si se requiere operar en sistemas con memoria restrictiva o entornos de alta concurrencia de caché, se debe emplear el **Binary Heap**. Si la memoria no es un factor limitante y se requiere unir dinámicamente múltiples colas de prioridad concurrentes, se debe adoptar el **Binomial Heap**.

6. Referencias

- [1] jfuentess. (s. f.). *edaa: Repositorio del curso Estructuras de datos y algoritmos avanzados* [Código fuente]. GitHub. <https://github.com/jfuentess/edaa>
- [2] Barbay, J., López-Ortiz, A., Lu, T., & Salinger, A. (2007). *Faster set intersection algorithms for text searching* (Technical Report CS-2007-13). University of Waterloo. <https://cs.uwaterloo.ca/research/tr/2007/CS-2007-13.pdf>
- [3] Bentley, J. L., & Yao, A. C.-C. (1976). An almost optimal algorithm for unbounded searching. *Information Processing Letters*, 5(3), 82–87. [https://doi.org/10.1016/0020-0190\(76\)90071-5](https://doi.org/10.1016/0020-0190(76)90071-5)
- [4] Demaine, E., Ku, J., & Solomon, J. (2020). *Lecture 8: Binary heaps* [Notas de clase del curso 6.006 Introduction to Algorithms]. Massachusetts Institute of Technology, MIT OpenCourseWare. https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/resources/mit6_006s20_lec8/
- [5] Sedgewick, R., & Wayne, K. (s. f.). *2.4 Priority queues*. Algorithms, 4th Edition. <https://algs4.cs.princeton.edu/24pq/>
- [6] National Institute of Standards and Technology. (s. f.). Binary heap. En *Dictionary of Algorithms and Data Structures*. <https://xlinux.nist.gov/dads/HTML/binaryheap.html>
- [7] ISO/IEC JTC1/SC22/WG21. (s. f.). Heap operations [alg.heap.operations]. En *Working draft, standard for programming language C++*. <https://eel.is/c++draft/alg.heap.operations>
- [8] ISO/IEC JTC1/SC22/WG21. (s. f.). Modifiers [vector.modifiers]. En *Working draft, standard for programming language C++*. <https://eel.is/c++draft/vector.modifiers>

6.1. Usos Específicos

Las implementaciones de referencia y las rutinas de medición de tiempo se tomaron del repositorio del curso [1].

El código fuente desarrollado, los scripts de automatización del banco de pruebas y los datasets generados para este boletín se encuentran alojados en el repositorio de reproducibilidad del proyecto: <https://github.com/StackDs/EDAA-Experiments>.

La descripción de la búsqueda galopante se fundamenta en los trabajos de Bentley y Yao y de Barbay et al. [3, 2].

El análisis de los heaps binarios se apoya en las notas del MIT, el material de Sedgewick y Wayne y el diccionario del NIST [4, 5, 6].

Las garantías de complejidad de las operaciones de biblioteca se consultaron en el borrador público del estándar de C++ [7, 8].

6.2. Declaración De Uso De IA Generativa

Durante la elaboración de este informe se utilizó ChatGPT, de OpenAI, como herramienta de apoyo para revisar la redacción académica, discutir el análisis de complejidad de los algoritmos y contrastar dicho análisis con las implementaciones estudiadas. De igual forma se utilizó Google Gemini para estructurar el repositorio con el código fuente y para mejorar los experimentos en términos de optimización. La responsabilidad por el contenido final, la interpretación de los resultados y la exactitud de las referencias corresponde al autor del informe.